

ORF307

**Solving linear systems in
practice**

Ioannis Akrotirianakis

Today's topics

- Matrices: definitions, operations, special cases
- Linear systems solutions
- Solving linear systems

-
- **Matrices**

Matrices

- **Notations:**

- $A \in \mathbb{R}^{m \times n}$
- There are n rows and m columns
- Contains mn real numbers as elements
- A_{ij} is the i, j elements
- A_{ij} **lives in** row i and column j
- **Vectors are matrices with only one column**

$$A = \begin{bmatrix} A_{11} & A_{12} & \cdots & A_{1n} \\ A_{21} & A_{22} & \cdots & A_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ A_{m1} & A_{m2} & \cdots & A_{mn} \end{bmatrix}$$

Special matrices

- **Zero matrix:** $A_{ij} = 0, \forall i, j$
- **Identity matrix:** $A_{ii} = 1, A_{ij} = 0, \forall i \neq j$
- **Sparse matrices:** most elements are zero
 - ❑ Special storage for memory savings
 - Values
 - Column index
 - Row pointer

Sparse matrix example

$$S = \begin{bmatrix} 7 & 3 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 2 & 0 & 3 \\ 0 & 8 & 5 & 0 & 0 & 0 \\ 0 & 0 & 6 & 0 & 4 & 0 \\ 0 & 0 & 0 & 0 & 12 & 0 \\ 0 & 14 & 0 & 9 & 0 & 0 \end{bmatrix}$$

$$vals = [7, 3, 1, 2, 3, 8, 5, 6, 4, 12, 14, 9]$$

$$colsIndex = [0, 1, 0, 3, 5, 1, 2, 2, 4, 4, 1, 3]$$

$$rowsPointer = [0, 2, 5, 7, 9, 10, 12]$$

Special matrices

- **Square matrix:** if $m = n$, $A \in R^{n \times n}$
- **Diagonal matrix:**
 - If it is square matrix and $A_{ij} = 0, \forall i \neq j$
 - **Notation:** $A = \text{diag}(a_1, a_2, \dots, a_n)$,

with $a_i = A_{ii}$

$$A = \text{diag}(-1, 2) = \begin{bmatrix} -1 & 0 \\ 0 & 2 \end{bmatrix}$$

- **Lower triangular matrix**

- $L \in R^{n \times n}$, $L_{ij} = 0, \forall j > i$

$$L = \begin{bmatrix} 1 & 0 & 0 \\ 2 & 3 & 0 \\ 4 & 5 & 6 \end{bmatrix}$$

- **Upper triangular matrix**

- $U \in R^{n \times n}$, $U_{ij} = 0, \forall i > j$

$$U = \begin{bmatrix} 1 & 2 & 3 \\ 0 & 4 & 5 \\ 0 & 0 & 6 \end{bmatrix}$$

Special matrices

- **Block matrices:** their elements are matrices themselves

- **Example 1:**

$$A = \begin{bmatrix} \textcolor{red}{B} & \textcolor{blue}{C} \\ \textcolor{green}{D} & \textcolor{brown}{E} \end{bmatrix} \in R^{(m_{\textcolor{red}{B}}+m_{\textcolor{green}{D}}) \times (n_{\textcolor{red}{B}}+n_{\textcolor{blue}{C}})},$$

$$F = \begin{bmatrix} \textcolor{red}{B} & \textcolor{blue}{c} \\ d^T & g \end{bmatrix} \in R^{(m_{\textcolor{red}{B}}+1) \times (n_{\textcolor{red}{B}}+1)}$$

- $\textcolor{red}{B} \in R^{m_{\textcolor{red}{B}} \times n_{\textcolor{red}{B}}}$, $\textcolor{blue}{C} \in R^{m_{\textcolor{red}{B}} \times n_{\textcolor{blue}{C}}}$, $\textcolor{green}{D} \in R^{m_{\textcolor{green}{D}} \times n_{\textcolor{red}{B}}}$, $\textcolor{brown}{E} \in R^{m_{\textcolor{green}{D}} \times n_{\textcolor{blue}{C}}}$
- $\textcolor{blue}{c} \in R^{m_{\textcolor{red}{B}}}$, $\textcolor{green}{d} \in R^{n_{\textcolor{red}{B}}}$, $g \in R$

- **Example 2:**

$$A = \begin{bmatrix} \textcolor{red}{B} & \textcolor{blue}{C} \\ \textcolor{green}{D} & \textcolor{brown}{E} \end{bmatrix} = \left[\begin{array}{ccc|cc} \textcolor{red}{0} & \textcolor{red}{1} & \textcolor{red}{2} & \textcolor{blue}{3} & \textcolor{blue}{4} \\ \textcolor{red}{5} & \textcolor{red}{6} & \textcolor{red}{7} & \textcolor{blue}{8} & \textcolor{blue}{9} \\ \hline \textcolor{green}{10} & \textcolor{green}{11} & \textcolor{green}{12} & \textcolor{brown}{13} & \textcolor{brown}{14} \\ \textcolor{green}{15} & \textcolor{green}{16} & \textcolor{green}{17} & \textcolor{brown}{18} & \textcolor{brown}{19} \\ \textcolor{green}{20} & \textcolor{green}{21} & \textcolor{green}{22} & \textcolor{brown}{23} & \textcolor{brown}{24} \\ \textcolor{green}{25} & \textcolor{green}{26} & \textcolor{green}{27} & \textcolor{brown}{28} & \textcolor{brown}{29} \end{array} \right]$$

$$A = \begin{bmatrix} \textcolor{red}{B} & \textcolor{blue}{c} \\ \textcolor{green}{d}^T & g \end{bmatrix} = \left[\begin{array}{ccc|c} \textcolor{red}{0} & \textcolor{red}{1} & \textcolor{red}{2} & \textcolor{blue}{3} \\ \textcolor{red}{5} & \textcolor{red}{6} & \textcolor{red}{7} & \textcolor{blue}{8} \\ \hline \textcolor{green}{10} & \textcolor{green}{11} & \textcolor{green}{12} & 13 \end{array} \right]$$

Special matrices

- **Column representation** of $A \in R^{m \times n}$

$$A = [a_1, a_2, \dots, a_n], \quad a_i \in R^m$$

- **Row representation** of $B \in R^{m \times n}$

$$B = \begin{bmatrix} b_1^T \\ b_2^T \\ \vdots \\ b_m^T \end{bmatrix}, \quad b_i \in R^n$$

Matrix operations

- **Transpose** of a matrix $A \in R^{m \times n}$
 - $(A^T)_{ij} = A_{ji}$
- **Addition** of two matrices $A, B \in R^{m \times n}$
 - $(A + B)_{ij} = A_{ij} + B_{ij}$
- **Scalar multiplication**
 - $(\gamma A)_{ij} = \gamma(A)_{ij}$ where $\gamma \in R$
- **More useful properties:**
 - $(A^T)^T = A$
 - $A + B = B + A$
 - $\gamma(A + B) = \gamma A + \gamma B$

Matrix-vector multiplication

- Consider $A \in R^{m \times n}$ and $x \in R^n$

$$y = Ax = (y_1, y_2, \dots, y_n)^T$$

where: $y_i = A_{i1}x_1 + A_{i2}x_2 + \dots + A_{in}x_n$

Example:

$$Ax = \begin{bmatrix} 0 & 2 & -1 \\ -2 & 1 & 1 \end{bmatrix} \begin{pmatrix} 2 \\ 1 \\ -1 \end{pmatrix} = \begin{pmatrix} 3 \\ -4 \end{pmatrix} = \begin{pmatrix} y_1 \\ y_2 \end{pmatrix}$$

Matrix-vector multiplication

- Consider $A \in R^{m \times n}$ and $x \in R^n$

$$y = Ax = (y_1, y_2, \dots, y_m)^T$$

where: $y_i = A_{i1}x_1 + A_{i2}x_2 + \dots + A_{in}x_n$

Example:

$$Ax = \begin{bmatrix} 0 & 2 & -1 \\ -2 & 1 & 1 \end{bmatrix} \begin{pmatrix} 2 \\ 1 \\ -1 \end{pmatrix} = \begin{pmatrix} 3 \\ -4 \end{pmatrix} = \begin{pmatrix} y_1 \\ y_2 \end{pmatrix}$$

- Row interpretation:**

$$y_i = A_i^T x, \quad i = 1, \dots, m$$

where A_i^T is the i -th row of A

Matrix-vector multiplication

- Consider $A \in R^{m \times n}$ and $x \in R^n$

$$y = Ax = (y_1, y_2, \dots, y_m)^T$$

where: $y_i = A_{i1}x_1 + A_{i2}x_2 + \dots + A_{in}x_n$

Example:

$$Ax = \begin{bmatrix} 0 & 2 & -1 \\ -2 & 1 & 1 \end{bmatrix} \begin{pmatrix} 2 \\ 1 \\ -1 \end{pmatrix} = \begin{pmatrix} 3 \\ -4 \end{pmatrix} = \begin{pmatrix} y_1 \\ y_2 \end{pmatrix}$$

- Row interpretation:**

$$y_i = A_i^T x, \quad i = 1, \dots, m$$

where A_i^T is the i -th row of A

- Column interpretation:**

$$y = a_1 x_1 + \dots + a_n x_n$$

where a_i is the i -th column of A

Linear combination of the columns

Return matrix in Portfolio Optimization

- Asset returns (in %) are stored in **matrix format**: $\mathcal{R} \in \mathbb{R}^{T \times n}$

← Number of assets
↑
Time periods

	NVDA	MSFT	GOOGL	AVGO
$\mathcal{R} =$	0.219	0.06	-0.113	0.202
	0.744	-0.894	-0.019	-0.468
	1.488	-0.215	0.433	-0.407
	$\vdots$	$\vdots$	$\vdots$	$\vdots$

$$\mathcal{R}_{tj} = \frac{p_{tj}^{final} - p_{tj}^{initial}}{p_{tj}^{initial}}$$

- $w = (w_1, w_2, w_3, w_4)^T$ is the allocation of your capital
- $r = \mathcal{R}w$ contains the **portfolio returns** over the periods 1 to T
- Example:** if $w = (0.4, 0.3, -0.2)^T$ then $\mathcal{R}w = (0.213, -0.201, 0.241)^T$ for $T = 3$.

Transpose

Symmetric Positive (Semi)Definite matrices

- **Symmetric matrix:**

- $A^T = A$

- **Positive Semidefinite, $\succcurlyeq$ (PSD)**

- $x^T A x \geq 0, \forall x \in R^n$

- **Positive Definite, $\succ$ (PD)**

- $x^T A x > 0, \forall x \in R^n, x \neq 0$

- **Example:**

Is the matrix $A = \begin{bmatrix} 2 & 6 \\ 6 & 20 \end{bmatrix} \succcurlyeq 0$ (PSD) ?

Take any $x \in R^2$ and compute:

$$(x_1 \quad x_2) \begin{bmatrix} 2 & 6 \\ 6 & 20 \end{bmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} =$$

$$= (x_1 \quad x_2) \begin{pmatrix} 2x_1 + 6x_2 \\ 6x_1 + 20x_2 \end{pmatrix} =$$

$$= 2x_1^2 + 12x_1x_2 + 20x_2^2 =$$

$$= 2(x_1 + 3x_2)^2 + 2x_2^2$$

$$\geq 0, \forall x \in R^2$$

Matrix multiplication

- Consider: $A \in R^{m \times p}$ and $B \in R^{p \times n}$

$$C = AB \in R^{m \times n}, \quad C_{ij} = A_{i1}B_{1j} + A_{i2}B_{2j} + \dots + A_{ip}B_{pj} = \sum_{k=1}^p A_{ik}B_{kj}$$

$$\left. \begin{array}{l} A = \begin{bmatrix} -1.5 & 3 & 2 \\ 1 & -1 & 0 \end{bmatrix} \in R^{3 \times 2} \\ B = \begin{bmatrix} -1 & -1 \\ 0 & -2 \\ 1 & 0 \end{bmatrix} \in R^{3 \times 2} \end{array} \right\} \begin{array}{l} \text{Move along the } i\text{-th row of } A \text{ and the } j\text{-th column of } B \\ C = AB = \begin{bmatrix} -1.5 & 3 & 2 \\ 1 & -1 & 0 \end{bmatrix} \begin{bmatrix} -1 & -1 \\ 0 & -2 \\ 1 & 0 \end{bmatrix} = \begin{bmatrix} 3.5 & -4.5 \\ -1 & 1 \end{bmatrix} \end{array}$$

Complexity

- For the matrices $A, B \in R^{m \times n}$, a vector $v \in R^n$, and a scalar $\gamma \in R$
 - $A + B$ and $\gamma A \rightarrow mn$ flops
 - $Av \rightarrow m(2n - 1) \approx 2mn$ flops
 - If $C \in R^{m \times p}$ and $D \in R^{p \times n}$, then $CD \rightarrow mn(2p - 1) \approx 2mnp$ flops
- **Questions/Exercises:**
 - How many flops does it take to multiply two 1000×10000 matrices?
 - How long does your computer need to do that? (Try it at home!)

-
- **Linear systems solutions**

Linear systems of equations

- **Given** a known matrix $A \in R^{m \times n}$ and vector $b \in R^m$, **find** $x \in R^n$:

$$Ax = b$$

Underdetermined (wide)

$$m < n$$

Infinite solutions

$$\begin{bmatrix} * & * & * & * \\ * & * & * & * \end{bmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} * \\ * \end{pmatrix}$$

Squared

$$m = n$$

Unique solution ($\exists A^{-1}$)

$$\begin{bmatrix} * & * \\ * & * \end{bmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} * \\ * \end{pmatrix}$$

Most common

Overdetermined (tall)

$$m > n$$

No solution

$$\begin{bmatrix} * & * \\ * & * \\ * & * \\ * & * \end{bmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} * \\ * \end{pmatrix}$$

Solving squared linear systems

- Given a known square matrix $A \in R^{n \times n}$ and vector $b \in R^n$, find $x \in R^n$:

$$Ax = b \Rightarrow A^{-1}Ax = A^{-1}b \Rightarrow x = A^{-1}b$$

- CAUTION:** A must be invertible ($\exists A^{-1}$)
 - When A is invertible ??
 - All columns of A are linear independent
 - All rows of A are linear independent
 - The columns or the rows of A form a **basis** of R^n

-
- **Solving systems of linear equations**

How to practically solve systems of linear equations

- Compute the inverse matrix A^{-1} and then compute $x = A^{-1}b$
- **Example:**
 - For a 10000×10000 real matrix A and a 10000 real vector b
 - You can use `numpy.linalg.inv` to compute $x = A^{-1}b$
 - You can use `numpy.linalg.solve` to compute $x = A^{-1}b$
 - **But what happens inside `numpy.linalg.solve`?**

Check GitHub Companion
for computational comparisons

Easy linear systems

- **Special case:** When the coefficient matrix $A \in R^{n \times n}$ is **diagonal**

$$Ax = b \Rightarrow \begin{bmatrix} A_{11} & 0 & \cdots & 0 \\ 0 & A_{22} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & A_{nn} \end{bmatrix} \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} = \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{pmatrix} \Rightarrow \text{diag}(A_{11}, A_{22}, \dots, A_{nn}) \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} = \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{pmatrix}$$

$$\Rightarrow \begin{array}{l} A_{11}x_1 = b_1 \\ A_{22}x_2 = b_2 \\ \vdots \\ A_{nn}x_n = b_n \end{array} \Rightarrow \begin{array}{l} x_1 = b_1/A_{11} \\ x_2 = b_2/A_{22} \\ \vdots \\ x_n = b_n/A_{nn} \end{array} \Rightarrow x = A^{-1}b$$

$$A^{-1} = \text{diag}(1/A_{11}, 1/A_{22}, \dots, 1/A_{nn})$$

Complexity: n flops

Easy linear systems

- **Lower triangular** coefficient matrix $A \in R^{n \times n}$

$$Ax = b \Rightarrow \begin{bmatrix} A_{11} & 0 & \cdots & 0 \\ A_{21} & A_{22} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ A_{n1} & A_{n2} & \cdots & A_{nn} \end{bmatrix} \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} = \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{pmatrix} \Rightarrow$$

Forward substitution

$$\begin{aligned} x_1 &= (b_1/A_{11}) \\ x_2 &= (b_2 - A_{21}x_1)/A_{22} \\ &\vdots \\ x_n &= (b_n - A_{n1}x_1 - \cdots - A_{n,n-1}x_{n-1})/A_{nn} \end{aligned}$$

Complexity:

- 1st equation: 1 flop
- 2nd equation: 3 flops

...

- n -th equation: $2(n - 1) + 1 = 2n - 1$

- **Total:** $1 + 3 + \cdots + 2n - 1 = n^2$ flops

Easy linear systems

- **Upper triangular** coefficient matrix $A \in R^{n \times n}$

$$Ax = b \Rightarrow \begin{bmatrix} A_{11} & A_{12} & \cdots & A_{1n} \\ 0 & A_{22} & \cdots & A_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & A_{nn} \end{bmatrix} \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} = \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{pmatrix}$$

Backward substitution

$$\begin{aligned} x_1 &= (b_1 - A_{12}x_2 - A_{13}x_3 - \cdots - A_{1n}x_n)/A_{11} \\ x_2 &= (b_2 - A_{23}x_3 - \cdots - A_{2n}x_n)/A_{22} \\ &\vdots \\ x_n &= b_n/A_{nn} \end{aligned}$$

Complexity:

- 1st equation: 1 flop
- 2nd equation: 3 flops

...

- n -th equation: $2(n - 1) + 1 = 2n - 1$

- **Total:** $1 + 3 + \cdots + 2n - 1 = n^2$ flops

Permutation matrices

Permutation matrices

- **Definition:** A matrix $P \in R^{n \times n}$ is called permutation matrix if it has exactly one entry of 1 in each row and column.
- When a permutation matrix multiplies a vector $x \in R^n$ it **reorders** the elements of the vector
- Define the desired permutation $\hat{x} = (x_{\pi_1}, x_{\pi_2}, \dots, x_{\pi_n})^T \in R^n$ of the original vector $x = (x_1, x_2, \dots, x_n)^T$.
 - Then compute matrix P and perform: $Px = \hat{x}$

Permutation matrices

Permutation matrices

- **Definition:** A matrix $P \in R^{n \times n}$ is called permutation matrix if it has exactly one entry of 1 in each row and column.
- When a permutation matrix multiplies a vector $x \in R^n$ it **reorders** the elements of the vector
- Define the desired permutation $\hat{x} = (x_{\pi_1}, x_{\pi_2}, \dots, x_{\pi_n})^T \in R^n$ of the original vector $x = (x_1, x_2, \dots, x_n)^T$.
 - Then compute matrix P and perform: $Px = \hat{x}$

$\pi = (2, 3, 1)^T$ is *permutation* of $(1, 2, 3)^T$

$$P_{ij} = \begin{cases} 1, & \text{if } j = \pi_i \\ 0, & \text{otherwise} \end{cases} \quad \text{for } i = 1, 2, 3$$

$$Px = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{bmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} x_2 \\ x_3 \\ x_1 \end{pmatrix} = \hat{x}$$

Property: $P^{-1} = P^T$ (Exercise)

$$P^{-1}\hat{x} = \begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix} \begin{pmatrix} x_2 \\ x_3 \\ x_1 \end{pmatrix} = \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = x$$

Permutation of rows and columns of a Matrix

- A permutation matrix can also change the rows or columns of another matrix

Row permutation: $\pi = (2,3,1)^T$

$$PA = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} A_{11} & A_{12} & A_{13} \\ A_{21} & A_{22} & A_{23} \\ A_{31} & A_{32} & A_{33} \end{bmatrix} = \begin{bmatrix} A_{21} & A_{22} & A_{23} \\ A_{31} & A_{32} & A_{33} \\ A_{11} & A_{12} & A_{13} \end{bmatrix}$$

Exercise:

Row & Column permutation:

$$\pi = (2,3,1)^T$$

Column permutation: $\pi = (2,3,1)^T$

$$AP^T = \begin{bmatrix} A_{11} & A_{12} & A_{13} \\ A_{21} & A_{22} & A_{23} \\ A_{31} & A_{32} & A_{33} \end{bmatrix} \begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix} = \begin{bmatrix} A_{12} & A_{13} & A_{11} \\ A_{22} & A_{23} & A_{21} \\ A_{32} & A_{33} & A_{31} \end{bmatrix}$$

$$PAP^T = ?$$

Importance of permutation matrices

- Permutations help us **rearrange** the rows and/or columns of the coefficient matrix.
 - Exchange a **zero or nearly zero** coefficient with a **non-zero** coefficient as **pivot**
 - Makes the solution of the linear system **possible and more stable**.

Example:

$$\begin{aligned}10^{-4}x_1 + x_2 + x_3 &= 1 \\ -x_1 + 2x_2 + 2x_3 &= 2 \\ 2x_1 + 2x_2 + 2x_3 &= 3\end{aligned}$$

- Without permutation we will use 10^{-4} as the first pivot.
- This will introduce **large numbers** in both the 2nd and 3rd rows.
- The factorization will become **inaccurate** (rounding off errors).

Permuted system

$$\begin{aligned}2x_1 + 2x_2 + 2x_3 &= 3 \\ -x_1 + 2x_2 + 2x_3 &= 2 \\ 10^{-4}x_1 + x_2 + x_3 &= 1\end{aligned}$$

- Rearranging 1st & 3rd columns we make the **largest coefficient** the **pivot**.
- Coefficient as the first pivot and the factorization becomes **more stable**.

What is the permutation matrix for this rearrangement??

Summary of easy linear systems $Ax = b$

- **Diagonal:** $A = \text{diag}(a_1, a_2, \dots, a_n)$

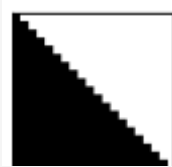
- Method: $x_i = b_i/a_i$



- Flops: n

- **Lower triangular:** $A_{ij} = 0, \forall j > i$

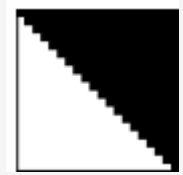
- Method: fwd substitution



- Flops: n^2

- **Upper triangular:** $A_{ij} = 0, \forall j < i$

- Method: bwd substitution



- Flops: n^2

- **Permutation:** $P_{ij} = 1, \text{ if } j = \pi_i; \text{ else } 0$

- Method: rearrangements

- Flops: 0

-
- **How to solve linear systems in practice?**

The factor-solve method for $Ax = b$

- **Step 1:** Factor the matrix A as a product of simpler matrices:

$$A = A_1 A_2 \dots A_k \quad \rightarrow \quad A_1 A_2 \dots A_k x = b$$

- A_i can be diagonal, lower or upper triangular, permutation $\rightarrow$ easier to solve intermediate systems

- **Step 2:** Solve $x = A^{-1}b = A_k^{-1} \dots A_2^{-1} A_1^{-1}b \quad \rightarrow$
 $A_1 y_1 = b \Rightarrow y_1 = A_1^{-1}b$
 $A_2 y_2 = y_1 \Rightarrow y_2 = A_2^{-1}y_1$
 $\vdots$

Note: step 1 is more computational expensive than step 2

$$A_k x = y_{k1} \Rightarrow x = A_1^{-1} y_{n-1}$$

Multiple right-hand sides

- Many times, we need to solve d linear systems with the **same coefficient matrix** $A \in R^{n \times n}$ but **different right-hand sides** ($b_1, b_2, \dots, b_d \in R^n$)

$$Ax = b_1, \quad Ax = b_2, \quad \dots, \quad Ax = b_d$$

- Factorize** A **once**
- Solve** all linear systems using the **same (cached) factorization**
 - The 'solve' can be done in **parallel**.

LU factorization

- Every invertible matrix A can be factored as

$$A = PLU \rightarrow P^T A = LU$$

Note: U is the upper triangular matrix of Gaussian elimination and the operations you perform are represented by the matrix L

- P : **permutation**, L : **lower triangular**, U : **upper triangular**.
- **Procedure:**
 - Similar to Gaussian elimination
 - Permutation P avoids divisions by 0
 - P, L, U may not be unique
- **Complexity:** $(2/3)n^3$ flops or $O(n^3)$

LU factorization

- Every invertible matrix A can be factored as

$$A = PLU \rightarrow P^T A = LU$$

Note: U is the upper triangular matrix of Gaussian elimination and the operations you perform are represented by the matrix L

- P : **permutation**, L : **lower triangular**, U : **upper triangular**.

- **Procedure:**

- Similar to Gaussian elimination
- Permutation P avoids divisions by 0
- P, L, U may not be unique

Note: We have to include P , because permutations allow us to avoid zero or nearly-zero coefficients and lets the factorization Process complete

- **Complexity:** $(2/3)n^3$ flops or $O(n^3)$

NOTE: Fewer flops if matrix A has **special structure** (e.g., lower/upper triangular or diagonal matrix)

LU factorization

- Every invertible matrix A can be factored as

$$A = PLU \rightarrow P^T A = LU$$

Note: U is the upper triangular matrix of Gaussian elimination and the operations you perform are represented by the matrix L

- P : permutation, L : lower triangular, U : upper triangular.

- **Procedure:**

- Similar to Gaussian elimination
- Permutation P avoids divisions by 0
- P, L, U may not be unique

Note: We have to include P , because permutations allow us to avoid zero or nearly-zero coefficients and lets the factorization Process complete

Historical information: a similar method was discovered In China many centuries earlier.

"The Nine Chapters on the Mathematical Art"

https://en.wikipedia.org/wiki/The_Nine_Chapters_on_the_Mathematical_Art

- **Complexity:** $(2/3)n^3$ flops or $O(n^3)$

NOTE: Fewer flops if matrix A has **special structure** (e.g., lower/upper triangular or diagonal matrix)

LU solution

- Once the factorization $A = PLU$ has been computed, we can solve the system:

$$Ax = b \Rightarrow PLUx = b \Rightarrow x = U^{-1}L^{-1}P^{-1}b$$

- STEP #1.** permutation: $Py_1 = b \Rightarrow y_1 = P^{-1}b$ (0 flops)
- STEP #2.** fwd subs: $Ly_2 = y_1 \Rightarrow y_2 = L^{-1}y_1$ (n^2 flops)
- STEP #3.** bwd subs: $Ux = y_2 \Rightarrow x = U^{-1}y_2$ (n^2 flops)
- Complexity:**
 - Factor+solve** = $(2/3)n^3 + 2n^2 \approx (2/3)n^3$

Cholesky (LL^T) factorization

- If $A \succcurlyeq 0$ is **PSD matrix** it can be **factorized** as

$$A = LL^T$$

- L is a **lower triangular** matrix
- **NOTE #1:** works only when A is symmetric PSD matrix
- **NOTE #2:** there is no need for permutation
- **Complexity:**
 - $(1/3)n^3$ flops (half of LU factorization)

Cholesky (LL^T) solution

- Once the LL^T factorization has been computed we can solve the system

$$\Rightarrow Ax = b \Rightarrow LL^T x = b \Rightarrow x = L^{-T} L^{-1} b$$

- Iterations:**

- STEP* #1. fwd subs: $Ly_1 = b \Rightarrow y_1 = L^{-1}b$ (n^2 flops)
- STEP* #2. bwd subs: $L^T x = y_1 \Rightarrow x = L^{-T}b$ (n^2 flops)

- Complexity:**

- Factor+Solve:** $(1/3)n^3 + 2n^2 \approx (1/3)n^3$

What complexity really means?

- **Example:** consider a matrix $A \in \mathbb{R}^{n \times n}$ with $n = 10,000$.
- Computational cost:
 - Factor+ Solve: $(2/3)n^3$
 - Only solve: $2n^2$
 - **Gains:** $\frac{(2/3)n^3}{2n^2} = \frac{1}{3}n$

Example: Polynomial interpolation (VMLS book, p.120)

- You have collected the **data** $D = \{d_0, d_1, d_2, d_3\} = \{(-1.1, b_0), (-0.4, b_1), (0.1, b_2), (0.8, b_3)\}$
- You want to interpolate the **cubic polynomial model**:

$$p(x) = c_0 + c_1x + c_2x^2 + c_3x^3$$

- We need to find the values of the coefficients c_0, c_1, c_2, c_3 such that:

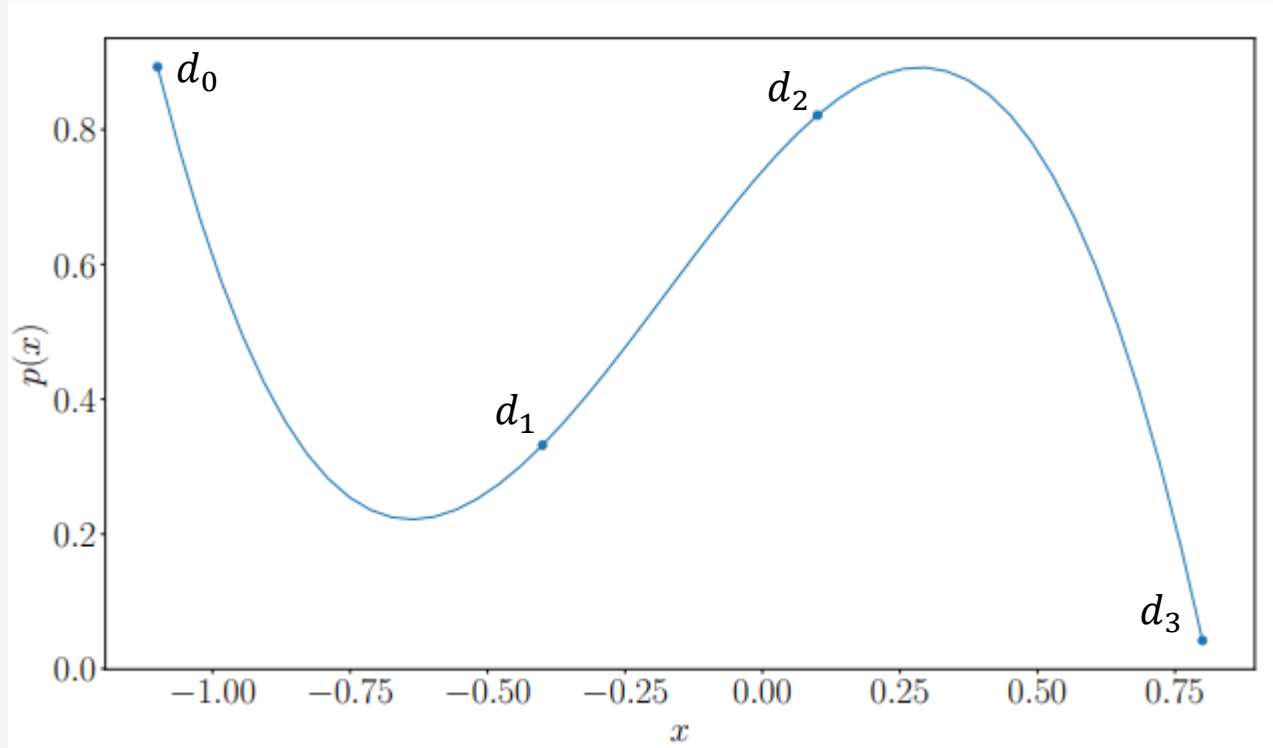
$$p(-1.1) = b_0, \quad p(-0.4) = b_1, \quad p(0.1) = b_2, \quad p(0.8) = b_3$$

- This results in solving the linear system:

$$Ac = b \Rightarrow \begin{bmatrix} 1 & -1.1 & (-1.1)^2 & (-1.1)^3 \\ 1 & -0.4 & (-0.4)^2 & (-0.4)^3 \\ 1 & 0.1 & (0.1)^2 & (0.1)^3 \\ 1 & 0.8 & (0.8)^2 & (0.8)^3 \end{bmatrix} \begin{pmatrix} c_0 \\ c_1 \\ c_2 \\ c_3 \end{pmatrix} = \begin{pmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \end{pmatrix}$$

Example – Polynomial interpolation

Solution: $c = (0.74, 0.93, 0.89, 1.70)^T$



Check GitHub Companion
for Python code on interpolation!

References

- S. Boyd, L. Vandenberghe: Introduction to Applied Linear Algebra – Vectors, Matrices, and Least Squares
 - Chapter 6: matrices
 - Chapter 10: matrix multiplication
 - Chapter 11: matrix inverses/solving linear systems
 - **Also, do not forget to check the Python notebooks at the Companion!**

Next lecture

- Solve optimization problems: Least Squares problems
